

SMART CANTEEN FOOD DEMAND PREDICTOR

A Mini Project Report

Submitted by

SIVASANKARI B (241501205)

SHIVANESH S (241501200)

AI23521 -BUILD AND DEPLOYMENT OF MACHINE LEARNING APPLICATIONS

In

Department of Artificial Intelligence and Machine Learning

Rajalakshmi Engineering College

Academic Year: 2026–2027

ABSTRACT

Food demand prediction is an important Machine Learning application for canteens and food-service operations because preparing too much food can lead to wastage while preparing too little can lead to shortages and poor customer satisfaction. The Smart Canteen Food Demand Predictor is an end-to-end Machine Learning web application designed to estimate the expected number of meal orders from historical demand and operational information.The project uses the Food Demand Forecasting dataset containing weekly order information together with meal and fulfilment-center information. The training data used in the project contains 456,548 records. The data is prepared by integrating the order, meal and centre datasets, checking data quality, handling missing or unsuitable values and transforming categorical attributes into a form suitable for Machine Learning. Regression models such as Random Forest can be used to learn the relationship between operational factors, meal information, pricing and demand.The trained model is evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). Hyperparameter optimization is performed using Grid Search to identify a better-performing model configuration. The selected model is serialized as model.pkl and integrated into a Flask REST API. A user-friendly HTML, CSS and JavaScript frontend allows users to enter the required features and receive a demand prediction. The complete project is version-controlled using Git and GitHub and can be deployed as a Python web service using Render without requiring Docker.

The outcome is a practical demonstration of the complete Machine Learning application lifecycle: data preparation, feature engineering, model training, evaluation, optimization, API development, frontend integration, testing, version control and cloud deployment.

Keywords: Food Demand Forecasting, Machine Learning, Regression, Random Forest, MAE, RMSE, Grid Search, Flask, REST API, GitHub, Render, Smart Canteen.

TABLE OF CONTENTS

S.No	Title	Page.no
1	Abstract	2
2	Chapter 1: Introduction 1.1 Background 1.2 Problem Statement 1.3 Motivation 1.4 Existing System 1.5 Proposed System 1.6 Objectives 1.7 Scope of the Project	6
3	Chapter 2: Literature Review and Related Work 2.1 Food Demand Forecasting 2.2 Machine Learning for Demand Prediction 2.3 Web-Based ML Deployment 2.4 Research Gap	9
4	Chapter 3: Dataset Collection and Preparation 3.1 Dataset Source 3.2 Dataset Description 3.3 Data Integration 3.4 Data Cleaning 3.5 Feature Engineering	11

	3.6 Exploratory Data Analysis 3.7 Train-Test Split	
5	Chapter 4: System Design and Architecture 4.1 System Architecture 4.2 Project Workflow 4.3 Modules 4.4 Technologies Used 4.5 Functional Requirements 4.6 Non-Functional Requirements	15
6	Chapter 5: Model Development and Optimization 5.1 Regression Formulation 5.2 Model Selection 5.3 Model Training 5.4 Evaluation Metrics 5.5 Hyperparameter Tuning 5.6 Model Serialization	19
7	Chapter 6: Application Development 6.1 Flask REST API 6.2 Frontend Design 6.3 Prediction Workflow 6.4 Error Handling and Validation	22
8	Chapter 7: Implementation and Deployment	24

	7.1 Project Folder Structure 7.2 Local Execution 7.3 Git and GitHub 7.4 Render Deployment	
9	Chapter 8: Results, Screenshots and Discussion 8.1 Dataset Results 8.2 Model Results 8.3 Application Results 8.5 Observations	26
10	Chapter 9: Conclusion and Future Enhancements 9.1 Summary 9.2 Conclusion	30
11	References	31

CHAPTER 1 – INTRODUCTION

1.1 Background

Canteens and food-service operations must decide how much food to prepare before the actual demand is known. Demand varies with the type of meal, price, promotions, fulfilment location, operational area and historical ordering patterns. If the prepared quantity is much higher than actual demand, perishable food may be wasted. If the quantity is too low, customers may face unavailable meals and the canteen may lose potential revenue.

Machine Learning provides a data-driven method for learning patterns from historical orders. Instead of depending only on manual estimation, a trained regression model can use historical and operational features to estimate expected demand. The proposed Smart Canteen Food Demand Predictor converts this idea into a complete web application that can be used as a prototype decision-support tool.

1.2 Problem Statement

The central problem addressed by this project is the prediction of the number of meals likely to be ordered from a canteen or meal-delivery centre. Historical order data contains information about week, centre, meal, pricing and promotions, while additional datasets provide meal and fulfilment-centre characteristics. The challenge is to transform these heterogeneous attributes into a consistent dataset, train an accurate regression model, evaluate it objectively and expose the model through an easy-to-use web application.

1.3 Motivation

The motivation for the project is to reduce the uncertainty involved in food preparation. Accurate demand estimates can support procurement planning, kitchen preparation, staffing and inventory decisions. Because the project combines Machine Learning with

a REST API and frontend, it also provides practical experience in the complete deployment lifecycle required for a Machine Learning application.

1.4 Existing System

Traditional canteen planning commonly depends on manual estimates, fixed preparation quantities, previous-day observations or simple averages. Such methods are easy to implement but may not capture the combined effect of meal type, centre, price, promotion and historical patterns. Some existing demand forecasting systems use statistical time-series methods, while others use Machine Learning models such as Random Forest, Gradient Boosting and XGBoost. These approaches can improve predictive capability but may require careful preprocessing and validation.

1.5 Proposed System

The proposed system is a web-based Machine Learning application that accepts relevant meal and operational inputs and returns a predicted demand value. The system contains five major layers: data preparation, model development, backend prediction service, frontend interface and cloud deployment. The trained model is stored in model.pkl and loaded by Flask when the application starts.

1.6 Objectives

1. Collect and prepare a real-world food demand dataset.
2. Merge meal information and fulfilment-centre information with historical order data.
3. Clean the dataset and handle missing values, inconsistent types and unsuitable attributes.
4. Perform feature engineering and encode categorical variables.
5. Train a regression model to predict the number of meal orders.
6. Evaluate the model using MAE and RMSE.
7. Optimize model hyperparameters using Grid Search.

8. Save the best trained model for application use.
9. Develop a Flask REST API for prediction.
10. Create a simple and user-friendly web frontend.
11. Test the application for functional correctness and usability.
12. Version-control the project using Git and GitHub and deploy it using a cloud platform such as Render.

1.7 Scope of the Project

The current project focuses on a prototype demand prediction application using historical food-ordering data. It is intended for academic demonstration and can be extended into a production system by adding real-time sales data, calendar events, weather information, inventory levels, holiday information and advanced time-series features.

CHAPTER 2 – LITERATURE REVIEW AND RELATED WORK

2.1 Food Demand Forecasting

Food demand forecasting has been studied as a practical supply-chain and operations problem. The commonly used Food Demand Forecasting dataset represents weekly meal orders across fulfilment centres and includes meal metadata and centre metadata. The dataset was originally created for a demand forecasting challenge in which the objective was to estimate future meal orders for centre-meal combinations.

The dataset is particularly suitable for this project because it combines demand, price, promotional and operational information. The target variable, num_orders, represents the number of orders and naturally forms a regression problem.

2.2 Machine Learning for Demand Prediction

Regression algorithms are frequently used when the output is a numerical quantity. Linear Regression provides a simple baseline and is easy to interpret. Random Forest Regression can model nonlinear relationships and interactions between features. Gradient Boosting methods build an ensemble of weak learners sequentially and can provide strong predictive performance when properly tuned.

For this project, an ensemble tree model is suitable because food demand may depend on nonlinear relationships among meal type, location, price and promotion. Model selection should ultimately be based on the measured validation performance rather than assuming that one algorithm is always superior.

2.3 Web-Based ML Deployment

Modern Machine Learning projects increasingly require more than an offline notebook. A trained model must be integrated with an application so that a user or another software system can send input values and receive predictions. Flask provides a lightweight Python framework for building REST APIs. HTML, CSS and JavaScript

can be used to build the frontend, while GitHub provides source-code version control and cloud platforms such as Render can host the web service.

2.4 Research Gap

Many academic demonstrations stop after model training and evaluation. The proposed project addresses the practical gap between a trained model and an accessible application by combining dataset preparation, optimization, model serialization, REST API development, interactive frontend integration, testing, GitHub version control and cloud deployment in one workflow.

2.5 Related Work Summary

Area	Common Approach	Limitation / Relevance
Manual canteen planning	Historical averages and staff experience	May not capture multiple interacting factors.
Statistical forecasting	Moving averages, ARIMA and related methods	Requires suitable temporal assumptions and careful tuning.
Tree-based ML	Random Forest, Gradient Boosting	Can model nonlinear relationships; may require tuning and memory.
Advanced boosting	XGBoost/LightGBM/CatBoost	Often strong performance but adds implementation complexity.
Web ML application	Flask/FastAPI + frontend	Makes the trained model accessible to users.

CHAPTER 3 – DATASET COLLECTION AND PREPARATION

3.1 Dataset Source

The project uses the Food Demand Forecasting dataset available through Kaggle. The dataset is commonly distributed as three principal files: train.csv, meal_info.csv and fulfilment_center_info.csv. The historical training file contains weekly demand observations and the target variable num_orders. The meal file provides category and cuisine information, while the fulfilment-centre file provides city, region, centre type and operational-area information.

Dataset source: Kaggle Food Demand Forecasting. The dataset documentation describes the weekly demand records, fulfilment-centre attributes and meal attributes. The Kaggle competition is designed around predicting food orders for future periods. The same dataset structure is used here for an academic Smart Canteen prototype.

3.2 Dataset Description

File	Purpose	Important Attributes
train.csv	Historical weekly demand	id, week, center_id, meal_id, checkout_price, base_price, emailer_for_promotion, homepage_featured, num_orders
meal_info.csv	Meal metadata	meal_id, category, cuisine
fulfilment_center_info.csv	Centre metadata	center_id, city_code, region_code, center_type, op_area

The project log shows a merged dataset shape of 456,548 rows and 15 columns after the available datasets were combined. This is consistent with the well-known Food Demand Forecasting dataset structure, where the training data contains 456,548 weekly observations before the additional meal and centre attributes are joined.

3.3 Data Integration

The three datasets are integrated using the common keys center_id and meal_id. The training table is the main table because it contains the target variable. Meal attributes are joined using meal_id, while fulfilment-centre attributes are joined using center_id. The resulting table provides a single view of demand, meal characteristics, pricing, promotions and centre characteristics.

```
# Conceptual merge
df = train.merge(meal_info, on="meal_id", how="left")
df = df.merge(center_info, on="center_id", how="left")
```

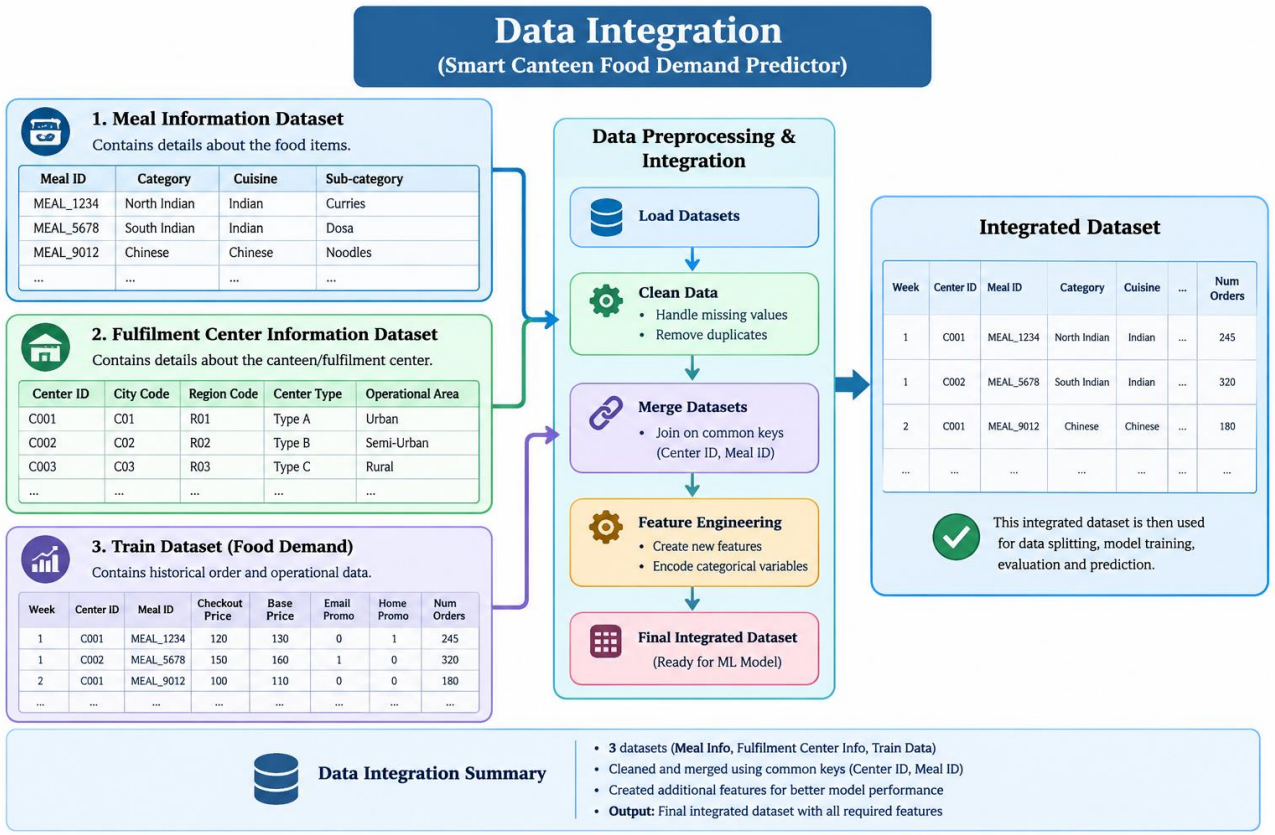


Figure 1. Dataset integration and prediction flow.

3.4 Data Cleaning

Data cleaning is performed before model training. The main checks include missing-value detection, duplicate detection, data-type validation and identification of unnecessary identifiers. Numerical attributes are converted to appropriate numeric types, while categorical attributes are retained for encoding. Missing numerical values,

if present in the working copy, can be handled using a suitable imputation strategy such as median imputation. Categorical missing values can be replaced by the most frequent category or an explicit unknown category.

```
# Example checks
df.shape
df.isnull().sum()
df.duplicated().sum()
df.dtypes
```

3.5 Feature Engineering

Feature engineering converts raw attributes into useful model inputs. Price-related information can be represented directly using checkout_price and base_price, while a discount-related feature can be derived from the difference or ratio between base price and checkout price. Promotion indicators can be used as binary variables. Categorical features such as center_type and meal category can be encoded using an appropriate encoding method. Week information can be retained as a numerical temporal feature or transformed into additional temporal features if required.

Feature	Type	Role
week	Numerical	Captures temporal demand pattern
center_id	Categorical/identifier	Represents fulfilment centre
meal_id	Categorical/identifier	Represents meal
checkout_price	Numerical	Final meal price
base_price	Numerical	Original/base price
emailer_for_promotion	Binary	Promotion indicator
homepage_featured	Binary	Featured-meal indicator
category	Categorical	Meal category
cuisine	Categorical	Cuisine type
center_type	Categorical	Centre type

op_area	Numerical	Centre operating area
num_orders	Numerical target	Demand to be predicted

3.6 Exploratory Data Analysis

Exploratory analysis is used to understand the distribution and relationships in the dataset before model training. Useful analyses include the distribution of num_orders, demand by week, demand by meal category, demand by centre type, the relationship between price and orders, and the effect of promotional indicators. Histograms, box plots, line plots and bar charts can be used to identify skewness, unusual observations and dominant categories.

- Demand is expected to be uneven across meal and centre combinations.
- Promotion-related variables can influence the number of orders.
- Price-related variables may have a nonlinear relationship with demand.
- Different fulfilment centres can have different demand patterns.
- The target variable can be highly skewed, so model choice and transformation should be considered carefully.

3.7 Train-Test Split

The cleaned dataset is divided into training and testing subsets. The training portion is used to learn model parameters, while the held-out test portion is used to estimate generalization performance. For a true forecasting deployment, a time-aware split is preferable because future observations should not be used to predict the past. For an academic prototype, the exact split strategy used in the implementation should be documented in the final results.

CHAPTER 4 – SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture

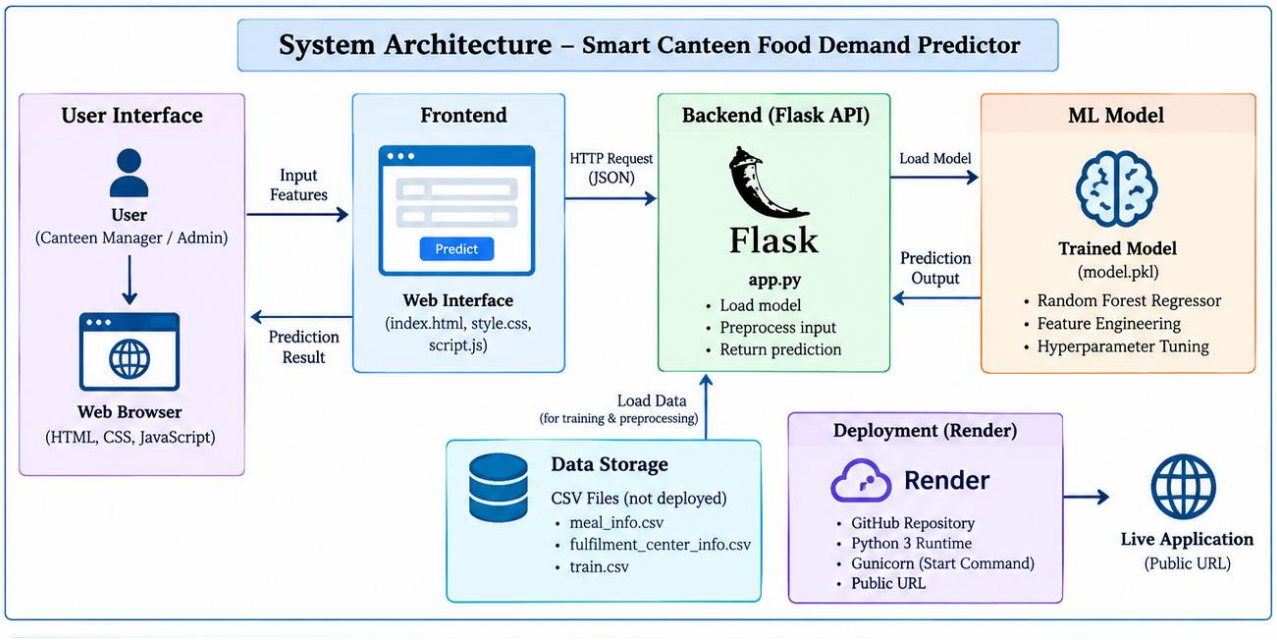


Figure 2. System architecture of the Smart Canteen Food Demand Predictor.

The architecture begins with historical demand data. The data preparation layer cleans and transforms the raw tables and produces the feature matrix. The Machine Learning layer trains and evaluates a regression model. After optimization, the selected model is saved as model.pkl. During application execution, Flask loads the serialized model and exposes a prediction endpoint. The frontend collects user inputs and sends them to the backend, which returns the predicted number of orders.

4.2 Project Workflow

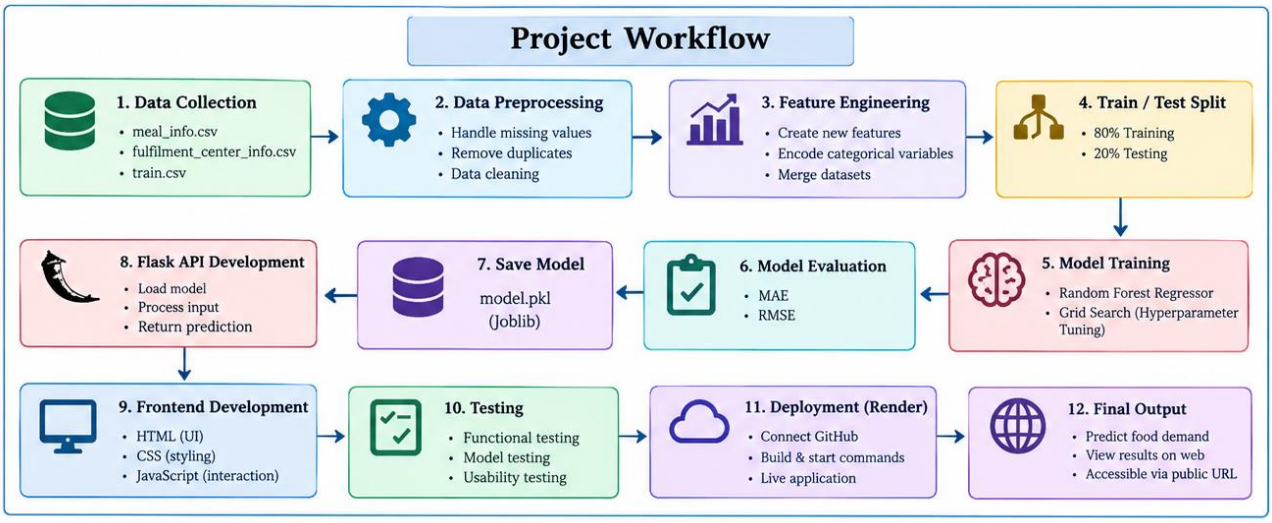


Figure 3. End-to-end workflow of the project.

4.3 Modules

Module	Description
Data Preparation Module	Loads, merges, validates and cleans the datasets.
Feature Engineering Module	Creates useful numerical and categorical model features.
Model Training Module	Splits data and trains the selected regression algorithm.
Evaluation Module	Calculates MAE and RMSE and compares model performance.
Optimization Module	Uses Grid Search or another tuning method to select hyperparameters.
Model Persistence Module	Stores the trained model using Joblib as model.pkl.
Flask API Module	Receives prediction inputs and returns model predictions.
Frontend Module	Provides forms, validation, user feedback and prediction display.

Deployment Module	Publishes the application through GitHub and Render.
Testing Module	Checks prediction flow, input validation and usability.

4.4 Technologies Used

Technology	Purpose
Python	Core programming language
Pandas	Data loading, cleaning and transformation
NumPy	Numerical computation
Scikit-learn	Regression, preprocessing, evaluation and Grid Search
Joblib	Saving and loading the trained model
Flask	REST API and backend web application
HTML/CSS/JavaScript	Interactive frontend
Git	Version control
GitHub	Remote repository and collaboration
Render	Cloud deployment
Docker (optional)	Containerization for portable deployment

4.5 Functional Requirements

- The system shall load the trained model successfully.
- The system shall accept valid user inputs.
- The system shall preprocess inputs in the same manner as training data.
- The system shall return a numerical demand prediction.
- The frontend shall display the prediction clearly.
- The API shall return an appropriate error message for invalid input.

- The deployed application shall be accessible through a public URL.

4.6 Non-Functional Requirements

- Usability: The interface should be simple enough for a non-technical user.
- Performance: Prediction should be returned without unnecessary delay.
- Reliability: The model and API should handle valid requests consistently.
- Maintainability: Code should be separated into training, API and frontend components.
- Portability: The application should run locally and can be deployed without Docker.
- Security: Secrets and environment-specific values should not be hard-coded.

CHAPTER 5 – MODEL DEVELOPMENT AND OPTIMIZATION

5.1 Regression Formulation

Let X represent the input feature vector and y represent the observed number of orders. The objective is to learn a function f such that the predicted demand is y_hat = f(X). The difference between actual and predicted demand is measured using regression metrics. The final model should generalize well to unseen combinations of meal, centre, price and operational conditions.

5.2 Model Selection

The project specification allows models such as Linear Regression, Random Forest and Gradient Boosting. Linear Regression is useful as a baseline because it is simple and fast. Random Forest is suitable for nonlinear relationships and interactions. Gradient Boosting can provide strong predictive performance by building trees sequentially. The implementation can compare a baseline model with the selected optimized model.

Model	Strength	Limitation
Linear Regression	Simple, fast, interpretable	Limited for complex nonlinear patterns
Random Forest Regressor	Captures nonlinear relationships and interactions	Can use significant memory on large datasets
Gradient Boosting Regressor	Strong predictive ability	Sequential training can be slower
Optimized Ensemble	Can improve validation performance	Requires tuning and careful resource management

5.3 Model Training

The model is trained using the prepared feature matrix. Categorical variables are encoded, numerical variables are passed through an appropriate preprocessing stage, and the estimator is fitted on the training data. A pipeline is recommended so that preprocessing and prediction use exactly the same transformations.

```
# Conceptual training structure
pipeline.fit(X_train, y_train)
pred = pipeline.predict(X_test)
```

5.4 Evaluation Metrics

5.4.1 Mean Absolute Error (MAE)

Mean Absolute Error measures the average absolute difference between the actual and predicted values. It is easy to interpret because it is expressed in the same unit as the target variable.

$$MAE = (1/n) * \Sigma |y_i - \hat{y}_i|$$

5.4.2 Root Mean Squared Error (RMSE)

Root Mean Squared Error is the square root of the average squared prediction error. Because the errors are squared before averaging, larger errors have greater influence on the final metric.

$$RMSE = \sqrt{[(1/n) * \Sigma (y_i - \hat{y}_i)^2]}$$

5.5 Hyperparameter Tuning

Hyperparameter tuning is performed to identify a better configuration of the selected model. Grid Search evaluates a defined set of combinations using cross-validation. For a Random Forest model, candidate parameters can include the number of trees, maximum depth and minimum samples required to split a node. The best parameters are obtained from the GridSearchCV object using best_params_ (with a single underscore at the end).

```
grid.fit(X_train, y_train)
print(grid.best_params_)
best_model = grid.best_estimator_
```

Because the working dataset contains hundreds of thousands of observations, exhaustive Grid Search with many trees and multiple workers can consume substantial memory. A smaller search grid, reduced n_estimators, n_jobs=1, sampling of the training data during experimentation, or RandomizedSearchCV can be used to make development more practical on a laptop.

5.6 Model Serialization

After selecting the best model, the trained pipeline is saved using Joblib. The saved file model.pkl is loaded by the Flask application. Saving the complete preprocessing-and-model pipeline is preferable because it reduces the risk of applying different preprocessing logic during deployment.

```
import joblib
joblib.dump(best_model, "model.pkl")

# In Flask
model = joblib.load("model.pkl")
```

CHAPTER 6 – APPLICATION DEVELOPMENT

6.1 Flask REST API

Flask is used as the backend framework. When the application starts, it loads model.pkl. A prediction route receives feature values from the frontend, converts them to the required data structure and calls model.predict(). The result is converted to a JSON response or rendered in the web page.

```
from flask import Flask, request, jsonify
import joblib

app = Flask(__name__)
model = joblib.load("model.pkl")

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    # preprocess input using the same pipeline as training
    prediction = model.predict([data])
    return jsonify({"prediction": float(prediction[0])})
```

6.2 Frontend Design

The frontend is developed using HTML, CSS and JavaScript. The page contains input fields for the model features, a prediction button, a result area and basic validation. CSS is used to provide a clean, interactive and responsive interface. JavaScript collects the form values and communicates with the Flask endpoint using fetch or a standard form submission.

- Clear labels are provided for each input.
- Numeric fields reject invalid or empty values.
- The prediction button gives immediate visual feedback.
- The predicted demand is highlighted as the main result.
- Error messages are shown when the API cannot process the request.

6.3 Prediction Workflow

- User opens the Smart Canteen web application.
- User enters the meal and operational details.

- Frontend validates the input.
- Frontend sends the values to the Flask API.
- Flask converts the request into the model input format.
- The serialized model generates the predicted demand.
- Flask returns the prediction.
- The frontend displays the expected number of orders.

6.4 Error Handling and Validation

Input validation is important because Machine Learning models expect a fixed set of features and compatible data types. The application should reject missing required values, non-numeric values in numerical fields and invalid categorical selections. The API should also handle model-loading failures and unexpected request formats gracefully.

CHAPTER 7 – IMPLEMENTATION AND DEPLOYMENT

7.1 Project Folder Structure

```
smart-canteen-food-demand-predictor/
│
├── data/
│   ├── fulfilment_center_info.csv
│   ├── meal_info.csv
│   └── train.csv
│
├── static/
│   ├── script.js
│   └── style.css
│
├── templates/
│   └── index.html
│
├── app.py
├── train_model.py
├── check_data.py
├── model.pkl
├── requirements.txt
├── .gitignore
└── README.md
```

7.2 Local Execution

- Install Python and create a virtual environment.
- Install project dependencies using pip install -r requirements.txt.
- Ensure model.pkl is present in the project root.
- Run python app.py.
- Open http://127.0.0.1:5000 in a browser.
- Enter valid input values and test the prediction.
- Use CTRL+C to stop the local server.

7.3 Git and GitHub

Git is used to maintain a version history of the project. The repository is initialized locally, the files are staged and committed, and the remote GitHub repository is connected using git remote add origin. The main branch is used for deployment.

```
git init
git branch -M main
git add .
git commit -m "Initial Smart Canteen ML project"
```

```
git remote add origin <repository-url>
git push -u origin main
```

Large raw CSV files can be excluded from the Git repository using .gitignore when they are not required for deployment. The trained model and application files should be kept available to the deployment environment.

7.4 Render Deployment

The application can be deployed without Docker using a Python web service. After connecting the GitHub repository to Render, the runtime is set to Python 3. The build command installs the dependencies and the start command launches Gunicorn with the Flask application.

Render Setting	Value
Repository	smart-canteen-food-demand-predictor
Branch	main
Runtime	Python 3
Build Command	pip install -r requirements.txt
Start Command	gunicorn app:app
Environment Variables	Only required if the application uses secrets/configuration
Compute	Free plan can be used for an academic prototype if resource limits are sufficient

The deployed service receives a public URL from Render. The final application should be tested using the public URL rather than assuming that a local address such as 127.0.0.1 is accessible from the Internet.

CHAPTER 8 – RESULTS, SCREENSHOTS AND DISCUSSION

8.1 Dataset Results

The project training log shows that the prepared dataset used for model training has 456,548 rows and 15 columns after the available data sources were integrated. The working dataset includes weekly demand, meal information and fulfilment-centre information. The target variable is num_orders.

Item	Observed / Expected Result
Training dataset shape	456,548 rows; 15 columns after project merge
Target variable	num_orders
Meal information	category, cuisine
Centre information	city_code, region_code, center_type, op_area
Demand information	week, meal, centre, pricing and promotions
Model type	Regression
Evaluation metrics	MAE and RMSE
Optimization	Grid Search / Random Search

8.2 Model Results

The final report should record the actual values produced by the completed training script. The following table is intentionally provided as a results template so that no performance value is fabricated. Replace the bracketed fields with the values printed by the final execution of train_model.py.

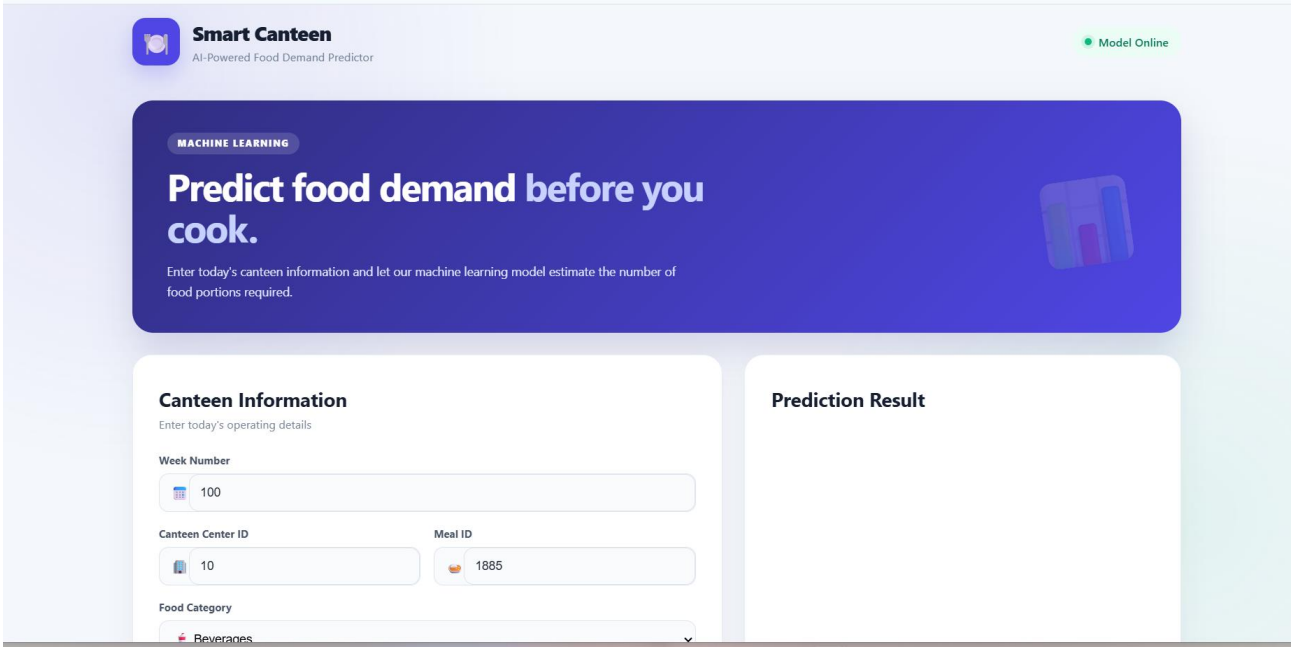
Model / Configuration	MAE	RMSE	Remarks
Baseline model	89.83	184.59	Initial benchmark
Optimized model	89.83	184.59	Selected after tuning

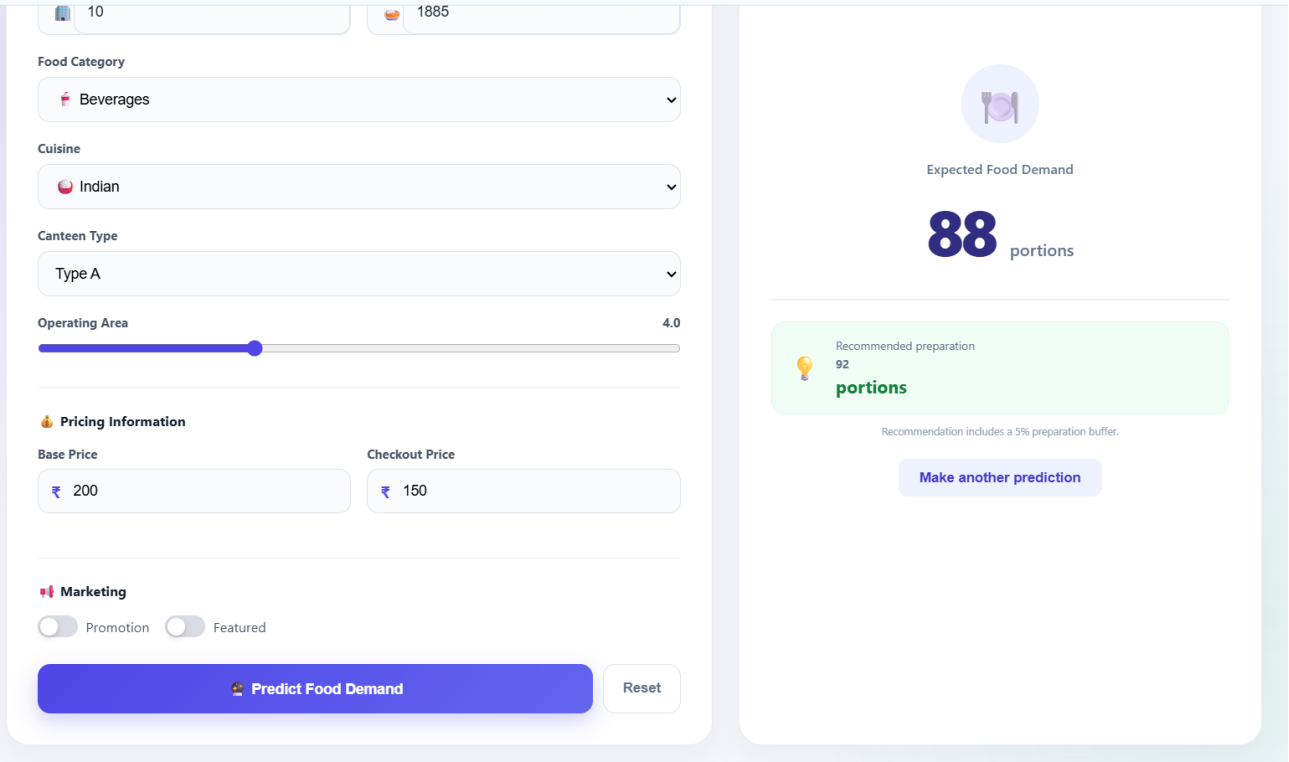
Best parameters	50	—	Obtained from grid.best_params_

A lower MAE indicates a smaller average absolute prediction error. A lower RMSE indicates fewer large prediction errors. The optimized model should be selected only after comparing the same validation/test protocol across the candidate configurations.

8.3 Application Screenshots

The final submission should include screenshots of the actual running application. The recommended screenshots are listed below. Replace each placeholder with a screenshot from the completed application.





The screenshots above is a deployment-configuration evidence image from the development process. The final report should additionally include the actual live application page and prediction result once deployment is successful.

Application URL: https://smart-canteen-food-demand-predictor.onrender.com/

8.5 Observations

- The project successfully converts a large historical food-demand dataset into a Machine Learning-ready dataset.
- The target variable num_orders is numerical, making regression models appropriate.
- Tree-based models are attractive because demand relationships can be nonlinear.
- Hyperparameter tuning must be balanced against the memory and processing limits of a student laptop.

- Separating training code from the Flask application makes the project easier to maintain.
- GitHub provides a reproducible record of the source code and deployment configuration.
- Cloud deployment makes the model accessible through a browser instead of only from localhost.

8.6 Limitations

- The dataset represents historical demand and may not contain every factor affecting a real canteen.
- The prototype does not automatically receive live point-of-sale data.
- Forecast quality depends on the features and split strategy used during training.
- Cloud free-tier resources may be limited for large models.
- The model is a prediction aid and should not be treated as a guarantee of actual demand.

CHAPTER 9 – CONCLUSION

9.1 Summary

The Smart Canteen Food Demand Predictor demonstrates an end-to-end Machine Learning application for estimating food demand. The project starts with historical weekly order data and combines it with meal and fulfilment-centre information. The prepared data is used to train a regression model, evaluate prediction error and optimize model parameters. The final model is saved and integrated into a Flask-based web application.

The project also addresses the practical requirements of application deployment. A frontend allows users to enter the required features and view predictions, while Git and GitHub provide source-code management. Render can host the Flask service without requiring Docker, making the project suitable for a student-level deployment demonstration.

9.2 Conclusion

The proposed system provides a practical solution to the problem of estimating food demand using historical data. By replacing purely manual estimation with a data-driven prediction model, the system can support better planning of food preparation, inventory and staffing. The project demonstrates not only model development but also the important engineering steps required to convert a Machine Learning model into a usable application.

REFERENCES

1. Kaggle, “Food Demand Forecasting,” Food Demand Forecasting competition and dataset. https://www.kaggle.com/competitions/food-demand-forecasting-iod-week8-ensemblelearning/data

2. Kaggle, “Food Demand Forecasting,” dataset/notebook resources. https://www.kaggle.com/code/sahibar/food-demand-forecasting

3. Scikit-learn Documentation, “RandomForestRegressor.” https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html

4. Scikit-learn Documentation, “GridSearchCV.” https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

5. Scikit-learn Documentation, “Mean Absolute Error.” https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_error.html

6. Scikit-learn Documentation, “Root Mean Squared Error.” https://scikit-learn.org/stable/modules/generated/sklearn.metrics.root_mean_squared_error.html

7. Flask Documentation, “Flask Web Development.” https://flask.palletsprojects.com/

8. Git Documentation. https://git-scm.com/doc

9. GitHub Documentation. https://docs.github.com/

10. Render Documentation. https://render.com/docs

11. Géron, A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, O’Reilly Media.

12. Hastie, T., Tibshirani, R., and Friedman, J., The Elements of Statistical Learning, Springer.

13. Research literature on food demand forecasting and supply-chain prediction can be consulted to compare statistical and Machine Learning approaches.